

Ex.No:12**MINIMIZATION OF DFA****PROGRAM:**

```
#include <stdio.h>

#define MAX 20

int n_states, n_symbols;

int transition[MAX][MAX];

int final_states[MAX], n_final;

int removed[MAX] = {0};

// Check if a state is final

int is_final(int s) {
    for (int i = 0; i < n_final; i++)
        if (final_states[i] == s) return 1;
    return 0;
}

// Check if two states are equivalent (same group + same transitions)

int equivalent(int s1, int s2) {
    if (is_final(s1) != is_final(s2)) return 0; // must belong to same group
    for (int j = 0; j < n_symbols; j++) {
        if (transition[s1][j] != transition[s2][j])
            return 0;
    }
    return 1;
}

// Replace all occurrences of old_state with new_state

void substitute_state(int old_state, int new_state) {
    for (int i = 0; i < n_states; i++) {
        for (int j = 0; j < n_symbols; j++) {
            if (transition[i][j] == old_state)
                transition[i][j] = new_state;
        }
    }
}
```

```

    }
}

int main() {
    int i, j;

    printf("Enter number of states: ");
    scanf("%d", &n_states);

    printf("Enter number of input symbols: ");
    scanf("%d", &n_symbols);

    printf("Enter transition table ( $\delta(q_i, \text{symbol}) = q_j$ ):\n");
    for (i = 0; i < n_states; i++) {
        for (j = 0; j < n_symbols; j++) {
            printf(" $\delta(q_{%d}, \%c) =$ ", i, 'a' + j);
            scanf("%d", &transition[i][j]);
        }
    }

    printf("Enter number of final states: ");
    scanf("%d", &n_final);

    printf("Enter final states: ");
    for (i = 0; i < n_final; i++)
        scanf("%d", &final_states[i]);

    // Step 1: Divide into groups and merge only inside groups
    for (i = 0; i < n_states; i++) {
        if (removed[i]) continue;
        for (j = i + 1; j < n_states; j++) {
            if (!removed[j] && equivalent(i, j)) {
                printf("q%d and q%d are equivalent (same group) → Eliminating q%d\n", i, j, j);
                substitute_state(j, i); // replace qj with qi
                removed[j] = 1;        // mark qj as removed
            }
        }
    }
}

```

```

// Step 2: Print minimized DFA

printf("\nMinimized DFA Transition Table:\n");

printf("State\t");

for (j = 0; j < n_symbols; j++)
    printf("%c\t", 'a' + j);

printf("\n");

for (i = 0; i < n_states; i++) {
    if (!removed[i]) {
        printf("q%d\t", i);

        for (j = 0; j < n_symbols; j++)
            printf("q%d\t", transition[i][j]);

        printf("\n");
    }
}

return 0;
}

```

OUTPUT:

```

Enter number of states: 4
Enter number of input symbols: 2
Enter transition table ( $\delta(q_i, \text{symbol}) = q_j$ ):
 $\delta(q_0, a) = 1$ 
 $\delta(q_0, b) = 2$ 
 $\delta(q_1, a) = 1$ 
 $\delta(q_1, b) = 2$ 
 $\delta(q_2, a) = 3$ 
 $\delta(q_2, b) = 3$ 
 $\delta(q_3, a) = 3$ 
 $\delta(q_3, b) = 3$ 
Enter number of final states: 2
Enter final states: 2 3
q0 and q1 are equivalent (same group) → Eliminating q1
q2 and q3 are equivalent (same group) → Eliminating q3

Minimized DFA Transition Table:
State    a      b
q0       q0     q2
q2       q2     q2

```

Ex.No:13**FINF FIRST AND FOLLOW****PROGRAM:**

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

int n, m = 0, i = 0, j = 0;

char a[10][10], f[10];

void follow(char c);

void first(char c);

int main() {

    int z;

    char c, ch;

    printf("Enter the number of productions:\n");

    scanf("%d", &n);

    printf("Enter the productions (example: E=E+T):\n");

    for (i = 0; i < n; i++) {

        scanf("%s", a[i]); // read production

    }

    do {

        m = 0;

        printf("\nEnter the element whose FIRST & FOLLOW is to be found: ");

        scanf(" %c", &c); // space before %c to ignore newline

        first(c);

        printf("FIRST(%c) = { ", c);

        for (i = 0; i < m; i++)

            printf("%c ", f[i]);

        printf("}\n");

        strcpy(f, " "); // clear buffer

        m = 0;

    } while (1);

}
```

```

    follow(c);

    printf("FOLLOW(%c) = { ", c);

    for (i = 0; i < m; i++)
        printf("%c ", f[i]);

    printf("}\n");

    printf("Continue? (1 = yes / 0 = no): ");

    scanf("%d", &z);

} while (z == 1);

return 0;

}

void first(char c) {
    int k;

    if (!isupper(c)) {
        f[m++] = c; // terminal directly in FIRST

        return;
    }

    for (k = 0; k < n; k++) {
        if (a[k][0] == c) {
            if (a[k][2] == '$') // epsilon production
                follow(a[k][0]);

            else if (islower(a[k][2])) // terminal
                f[m++] = a[k][2];

            else
                first(a[k][2]); // non-terminal
        }
    }
}

void follow(char c) {
    if (a[0][0] == c) // start symbol
        f[m++] = '$';

```

```

for (i = 0; i < n; i++) {
    for (j = 2; j < strlen(a[i]); j++) {
        if (a[i][j] == c) {
            if (a[i][j + 1] != '\0')
                first(a[i][j + 1]);
            if (a[i][j + 1] == '\0' && c != a[i][0])
                follow(a[i][0]);
        }
    }
}
}

```

OUTPUT:

```

Enter the number of productions:
5
Enter the productions (example: E=E+T):
S=AbCd
A=Cf
A=a
C=gE
E=h

Enter the element whose FIRST & FOLLOW is to be found: S
FIRST(S) = { g a }
FOLLOW(S) = { $ }
Continue? (1 = yes / 0 = no): 0

```